DEVPILOT AI

API Specification

Technical Documentation

REST API contract for the approved DevPilot AI business flow: Authentication, Email Verification, Problems, AI Debugging, Knowledge Base, Similarity Search, Challenges, GitHub, Code Review, Telegram, AI Gateway, and Administration.

Version	1.0
Backend	ASP.NET Core Web API
Database	SQL Server
Backend Hosting	MonsterASP
Frontend	Undefined yet
Frontend Hosting	Cloudflare Pages
Email Provider	Brevo

Contents

Architecture Overview 4
Platform Summary 4
Access Model 5
1. General Rules 6
Generic Error Response 6
2. Authentication & Email Verification 7
Register – Request 7
Register – Process 8
Verify Email – Requests 8
Security 8
3. User / Profile 9
4. Problems & Debugging 10
Create Problem – Request 10
Debug – Processing 11
Debug – Response Example 11
Feedback 12
5. Search & Similarity 13
Search Query Parameters 13
Similarity Signals 13
6. Knowledge Articles 14
Article Response Contents 14
Article Versioning 14
7. Challenges 15
Start Challenge – Process 15
Evaluation Criteria 15
8. GitHub 16
Repository Debug – Request 16
9. AI Code Review 17
Request 17
10. Telegram 18
11. AI Gateway 19
Generate – Request 19
Gateway Responsibilities 19
Provider Behavior 20
12. Admin 21

Possible Article Actions 21
13. Status / Enums 22
14. HTTP Status Codes 23
15. Security Requirements 24
16. Token Optimization Rules 25
17. Article Evolution Rules 26
18. GitHub Challenge Flow 27
19. API Implementation Order 28
20. Final API Flow 29

Architecture Overview

DevPilot AI is delivered as an ASP.NET Core Web API hosted on MonsterASP, backed by SQL Server. The frontend (technology not yet decided) is served from Cloudflare Pages and communicates with the API over JSON/HTTPS. The API integrates with Brevo for email, GitHub for repositories and webhooks, Telegram for the bot, and multiple LLM providers through an internal AI Gateway.

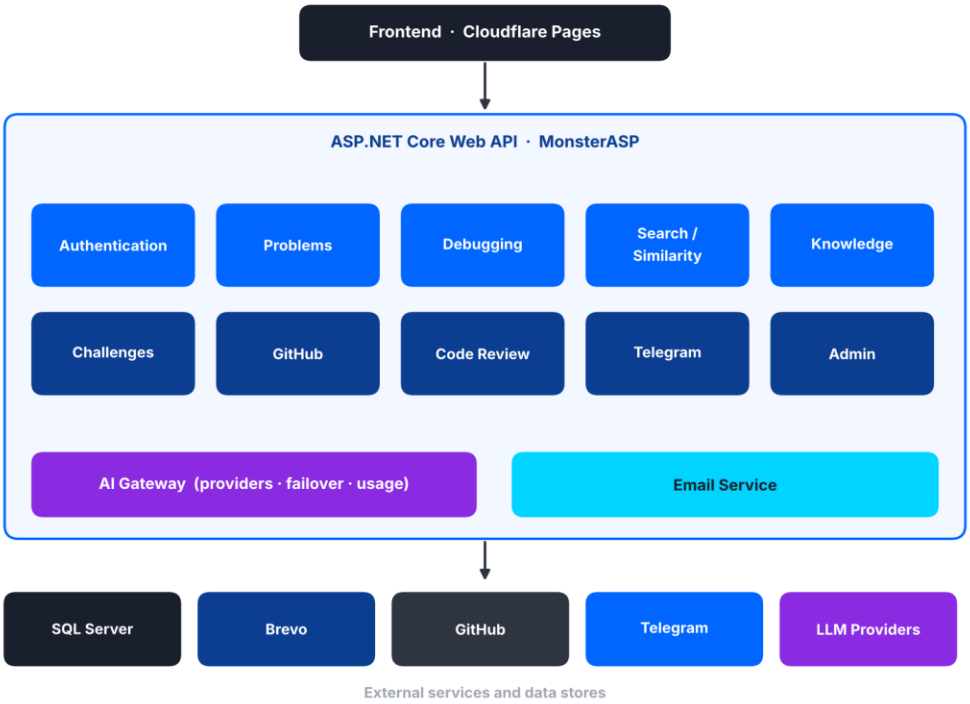


Figure 1 – System architecture and final API flow

Platform Summary

Layer	Technology / Role
Backend	ASP.NET Core Web API – all business logic, authentication, orchestration
Database	SQL Server – users, problems, attempts, articles, challenges, AI usage
Backend hosting	MonsterASP (Docker is not part of this hosting layer)
Frontend	Not yet defined
Frontend hosting	Cloudflare Pages (Cloudflare Turnstile used for abuse protection)
Email	Brevo – verification and password-reset emails
External services	GitHub (OAuth, repositories, webhooks), Telegram (bot webhook), LLM providers (via AI Gateway)

Access Model

Three access levels apply across the API. Public endpoints include article browsing/search and selected integration webhooks. Protected endpoints require a bearer access token. Admin endpoints require Administrator authorization.

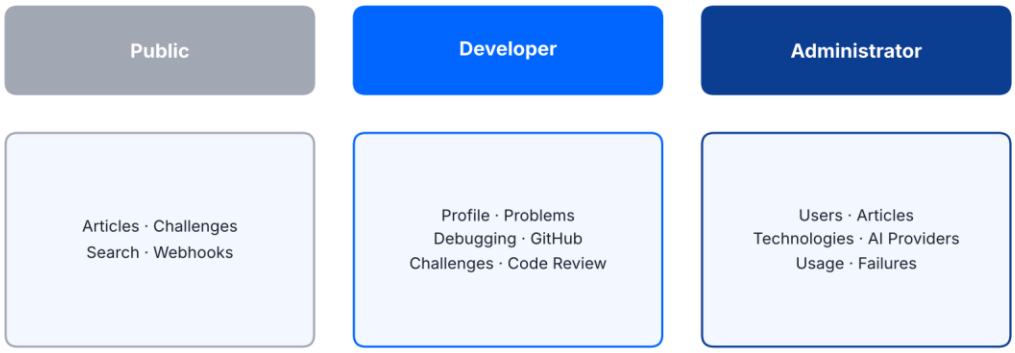


Figure 2 – Access levels and responsibilities

1. General Rules

Rule	Value
Base path	/api
Format	JSON over HTTPS
Authentication	Bearer access token for protected endpoints
Roles	Developer, Administrator

Generic Error Response

```
{
  "success": false,
  "errorCode": "ERROR_CODE",
  "message": "Safe user-facing message",
  "requestId": "correlation-id"
}
```

2. Authentication & Email Verification

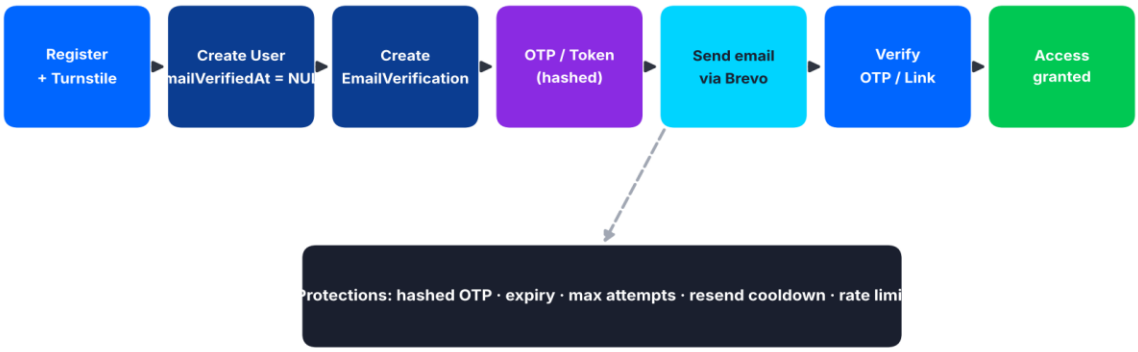


Figure 3 – Registration and email verification flow

Method	Endpoint	Access	Purpose
POST	`/api/auth/register`	Public	Create developer account and start email verification
POST	`/api/auth/verify-email`	Public	Verify OTP or verification token
POST	`/api/auth/resend-verification`	Public	Invalidate active verification and send a new OTP/link through Brevo
POST	`/api/auth/login`	Public	Authenticate verified developer
POST	`/api/auth/refresh`	Refresh token	Issue new access token
POST	`/api/auth/logout`	Authenticated	End session
POST	`/api/auth/forgot-password`	Public	Send reset email through Brevo without revealing account existence
POST	`/api/auth/reset-password`	Public (valid reset token)	Reset password

Register – Request

```
{
  "name": "Mahmoud",
  "email": "developer@example.com",
  "password": "StrongPassword",
  "turnstileToken": "..."
}
```

Register – Process

1. Validate input.
2. Create User with EmailVerifiedAt = NULL.
3. Create EmailVerification record.
4. Generate OTP or verification token.
5. Send verification email through Brevo.
6. Return verification-required result.

Verify Email – Requests

OTP request:

```
{
  "email": "developer@example.com",
  "code": "483921"
}
```

Link request:

```
{
  "token": "verification-token"
}
```

On success:

- Mark EmailVerifications.VerifiedAt.
- Set Users.EmailVerifiedAt.
- Invalidate active verification.
- Allow authenticated access.

Security

- Store OTP/token hashes, not raw OTP values.
- Apply expiration.
- Apply maximum attempts.
- Apply resend cooldown/rate limiting.
- Never expose Brevo credentials to frontend.

3. User / Profile

Method	Endpoint	Access	Purpose
GET	`/api/users/me`	Developer	Returns current developer profile
PATCH	`/api/users/me`	Developer	Updates profile
GET	`/api/users/me/problems`	Developer	Returns current developer's problem history
GET	`/api/users/me/challenges`	Developer	Returns challenge attempts and results
GET	`/api/users/me/integrations`	Developer	Returns integration status

4. Problems & Debugging

Method	Endpoint	Access	Purpose
POST	`/api/problems`	Developer	Create a new technical problem
POST	`/api/problems/{problemId}/requests`	Developer	Add another request/input while the problem is unresolved
POST	`/api/problems/{problemId}/debug`	Developer	Run AI debugging
POST	`/api/problems/{problemId}/attempts/{attemptId}/feedback`	Problem owner	Submit result of an attempt
GET	`/api/problems/{problemCode}`	Owner or public view	Owner, or public article/knowledge view per visibility rules
GET	`/api/problems/{problemId}/history`	Owner	Requests, attempts, feedback, successful solutions

Create Problem – Request

```
{
  "title": "JWT returns 401",
  "description": "...",
  "errorMessage": "401 Unauthorized",
  "stackTrace": "...",
  "technologyHint": "ASP.NET Core"
}
```

Debug – Processing

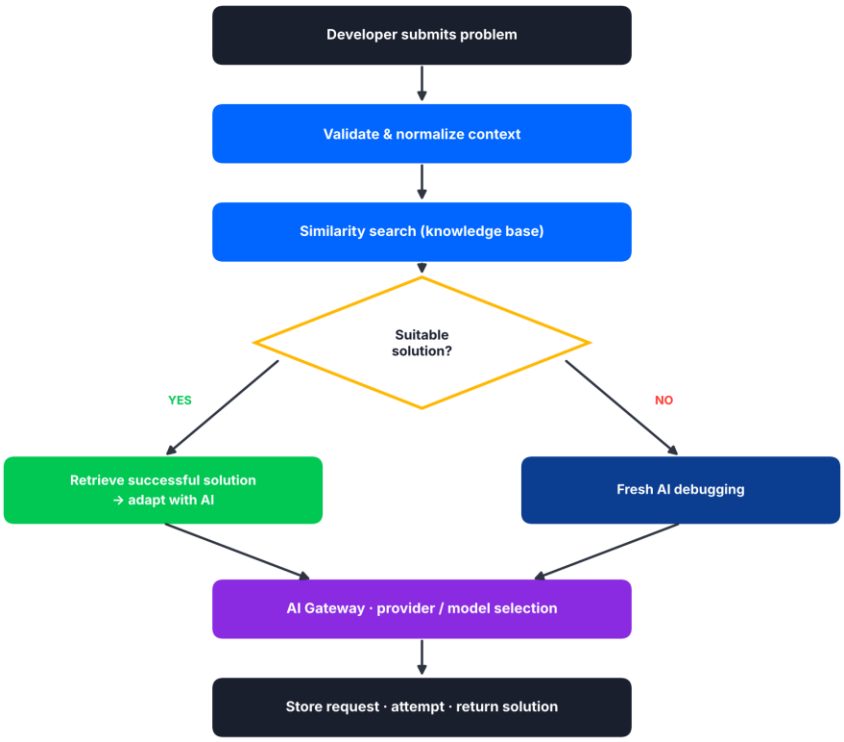


Figure 4 – Debugging decision flow (similarity first, AI second)

1. Validate input.
2. Normalize technical context.
3. Search similar historical problems.
4. Evaluate similarity.
5. Retrieve successful reusable solutions when appropriate.
6. Build optimized AI context.
7. Call AI Gateway.
8. Select available provider/model.
9. Validate/normalize response.
10. Store AI request/response.
11. Store DebuggingAttempt.
12. Return solution.

Debug – Response Example

```
{
  "success": true,
  "data": {
    "attemptId": 12,
    "attemptNumber": 1,
```

```
    "rootCause": "...",
    "solution": "...",
    "whyItWorks": "...",
    "similarProblems": [
      {
        "problemCode": "DEV-000102",
        "similarityScore": 0.94
      }
    ]
  }
}
```

Feedback

Request:

```
{
  "result": "Solved",
  "comment": "..."
}
```

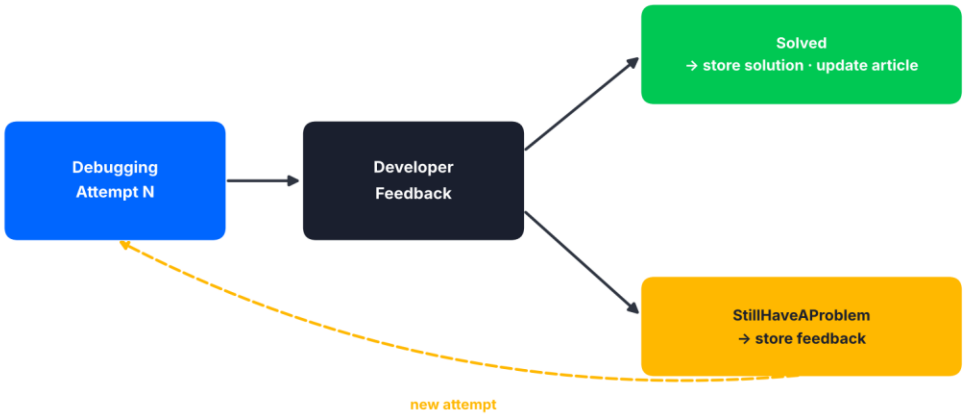


Figure 5 – Feedback outcomes and retry loop

Result	Behavior
Solved	Mark problem solved. Store successful solution. Update knowledge/article workflow.
StillHaveAProblem	Store feedback. Start another debugging attempt.

5. Search & Similarity

Method	Endpoint	Access	Purpose
GET	`/api/problems/search`	Public / Protected (configured visibility)	Search problems
POST	`/api/search/similar-problems`	Authenticated	Semantic/keyword search before unnecessary LLM processing

Search Query Parameters

Parameter	Description
`q`	Free-text query
`technology`	Technology filter
`tag`	Tag filter
`problemCode`	Exact problem code
`error`	Error message text
`page`	Page number
`pageSize`	Items per page
`sort`	Sort order

Similarity Signals

The similarity system considers:

- Problem description
- Error messages
- Stack traces
- Code context
- Technology/framework
- Tags
- Previously successful solutions

6. Knowledge Articles

Method	Endpoint	Access	Purpose
GET	`/api/articles`	Public	List articles
GET	`/api/articles/{articleId}`	Public	Get article
GET	`/api/articles/{articleId}/versions`	Administrator	List versions
GET	`/api/articles/{articleId}/versions/{versionId}`	Administrator	Get a specific version
POST	`/api/problems/{problemId}/article/generate`	Internal workflow / Administrator	Generate article from problem
POST	`/api/articles/{articleId}/regenerate`	Internal workflow / Administrator	Regenerate article

Article Response Contents

- Problem code
- Problem
- Error/code context
- Root cause
- Step-by-step solution
- Explanation of each step
- Best practices
- Prevention
- Statistics
- Related problems
- Practice challenge when available

Article Versioning

- New successful knowledge can generate a new ArticleVersion.
- Previous versions are preserved.
- Current article points to CurrentVersionId.

7. Challenges

Method	Endpoint	Access	Purpose
GET	`/api/challenges`	Public	List challenges
GET	`/api/challenges/{challengeId}`	Public	Get challenge (never expose the complete solution)
POST	`/api/challenges`	Authenticated or internal AI workflow	Create/generate challenge from a solved problem
POST	`/api/challenges/{challengeId}/start`	Developer	Start a challenge attempt
GET	`/api/challenges/attempts/{attemptId}`	Attempt owner	Get attempt
POST	`/api/challenges/attempts/{attemptId}/evaluate`	Attempt owner / internal workflow	Evaluate attempt

> **Important**
> GET /api/challenges/{challengeId} must not expose the complete solution.

Start Challenge – Process

1. Verify authentication.
2. Create ChallengeAttempt.
3. Create/prepare GitHub repository when permitted.
4. Associate repository with attempt.
5. Return challenge repository/environment information.

Evaluation Criteria

- Build/test results
- Functional correctness
- Requirement completion
- Code quality
- Security
- Performance
- Architecture
- Understanding of original problem
- Best practices

8. GitHub

Method	Endpoint	Access	Purpose
GET	`/api/github/connect`	Developer	Starts OAuth
GET	`/api/github/callback`	OAuth callback	Must validate state
GET	`/api/github/repositories`	Developer	Repositories available through GitHub permissions
GET	`/api/github/repositories/{repositoryId}/files`	Developer	Retrieve only relevant repository context (query: branch, path)
POST	`/api/github/repositories/{repositoryId}/debug`	Developer	Debug using selected repository files
POST	`/api/github/challenges/{challengeId}/repository`	Developer	Create/initialize the challenge repository
POST	`/api/github/webhook`	GitHub only	Receive Push and Pull Request events; must validate authenticity

Repository Debug – Request

```
{
  "branch": "main",
  "paths": ["Program.cs", "AuthService.cs"],
  "problemDescription": "JWT returns 401"
}
```

Token efficiency

The API should avoid sending unnecessary files to the LLM.

9. AI Code Review

Method	Endpoint	Access	Purpose
POST	`/api/code-review`	Developer	Start a code review
GET	`/api/code-review/{reviewId}`	Review owner / Administrator	Get review and findings

Request

```
{
  "repositoryId": 55,
  "branchName": "main",
  "pullRequestNumber": 18,
  "files": ["src/Services/AuthService.cs"]
}
```

Review may identify	Finding response includes
Bugs	FilePath
Security issues	Severity
Performance issues	Category
Code smells	Problem
SOLID violations	Suggestion
Maintainability	
Architecture concerns	
Best practices	

10. Telegram

Method	Endpoint	Access	Purpose
POST	`/api/telegram/webhook`	Telegram	Receive bot messages
GET	`/api/telegram/status`	Developer	Integration status

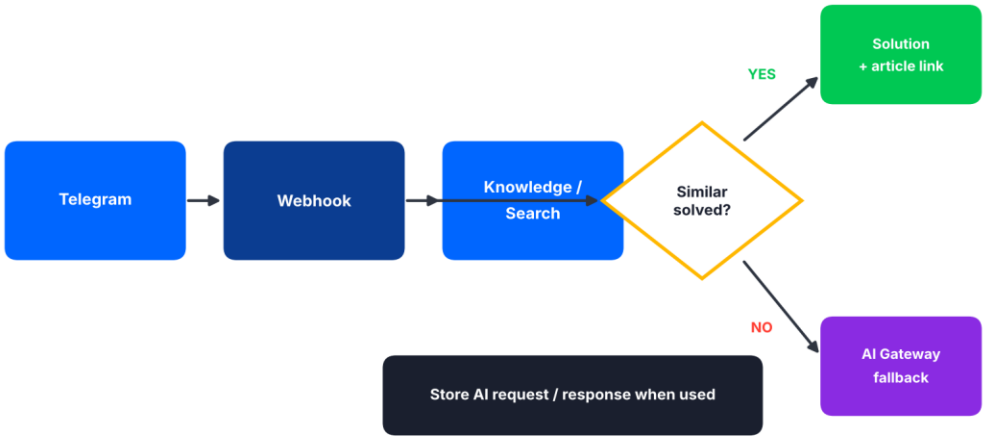


Figure 6 – Telegram answer flow

11. AI Gateway

> **Internal**
> These endpoints are internal and should not be publicly exposed.

Method	Endpoint	Access	Purpose
POST	`/api/ai/generate`	Internal	Central AI operation entry point
GET	`/api/admin/ai/providers/health`	Administrator	Provider health
GET	`/api/admin/ai/providers`	Administrator	List providers
POST	`/api/admin/ai/providers`	Administrator	Add provider
PATCH	`/api/admin/ai/providers/{providerId}`	Administrator	Update provider
GET	`/api/admin/ai/usage`	Administrator	Usage
GET	`/api/admin/ai/failures`	Administrator	Failures

Generate – Request

```
{
  "operationType": "Debugging",
  "problemId": 1001,
  "context": "..."
}
```

Gateway Responsibilities

- Provider selection
- Model selection
- Usage tracking
- Retry
- Failover
- Response normalization
- Prompt version selection

Provider Behavior

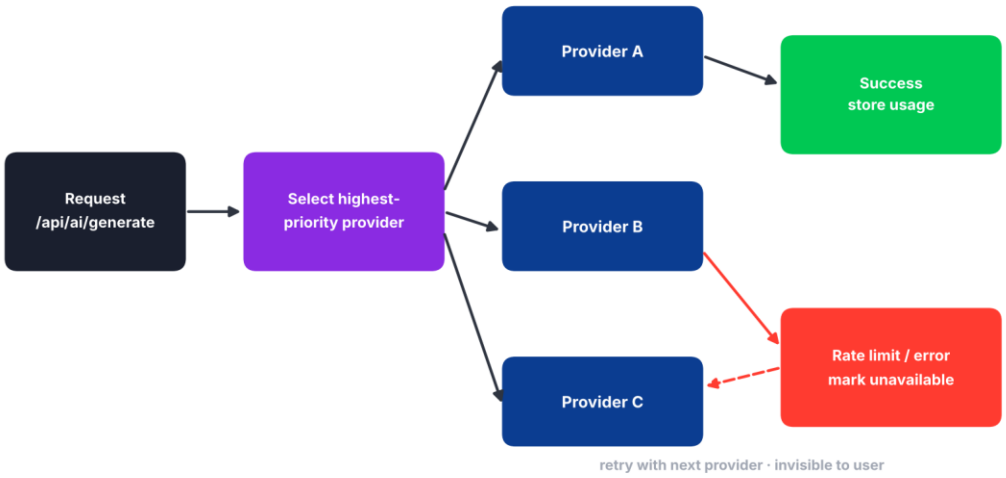


Figure 7 – Provider selection and failover

1. Select highest-priority available provider/model.
2. Call provider.
3. On success, store usage.
4. On rate limit/error, mark provider/model temporarily unavailable.
5. Retry with next available provider.
6. Do not expose internal switching to user.

> **Credentials**
> Provider credentials must never be returned.

12. Admin

Method	Endpoint	Access	Purpose
GET	`/api/admin/users`	Administrator	List users
GET	`/api/admin/users/{userId}`	Administrator	Get user
PATCH	`/api/admin/users/{userId}`	Administrator	Update user
GET	`/api/admin/problems`	Administrator	List problems
GET	`/api/admin/problems/{problemId}`	Administrator	Get problem
GET	`/api/admin/articles`	Administrator	List articles
PATCH	`/api/admin/articles/{articleId}`	Administrator	Article actions
GET	`/api/admin/technologies`	Administrator	List technologies
POST	`/api/admin/technologies`	Administrator	Add technology
PATCH	`/api/admin/technologies/{technologyId}`	Administrator	Update technology

Possible Article Actions

- Approve
- Hide
- Request regeneration
- Change publication state

13. Status / Enums

Enum	Values
`VerificationMethod`	OTP, Link
`ProblemStatus`	Active, Solved
`FeedbackResult`	Solved, StillHaveAProblem
`ChallengeDifficulty`	Easy, Medium, Hard
`ChallengeAttemptStatus`	Started, Submitted, Evaluating, Evaluated, Completed, Failed
`AIRequestStatus`	Pending, Success, Failed, RateLimited, Cancelled
`ReviewStatus`	Pending, Analyzing, Completed, Failed
`ArticleStatus`	Draft, Published, Archived, NeedsReview

14. HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Entity
429	Too Many Requests
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

15. Security Requirements

- HTTPS only.
- Secure authentication and authorization.
- Validate all request data.
- Rate limit abuse-sensitive endpoints.
- Use Cloudflare Turnstile where appropriate.
- Never expose AI provider API keys to frontend.
- Never expose GitHub tokens to frontend.
- Store GitHub credentials securely/encrypted.
- Store OTP/token as hashes.
- Expire verification/reset records.
- Apply resend cooldown.
- Validate GitHub webhook authenticity.
- Use least-privilege GitHub permissions.
- Keep private repository data private.
- Do not place sensitive credentials inside AI prompts.
- Centralized exception handling.
- Never return internal stack traces in production.

16. Token Optimization Rules

Every new request is normalized and checked against the knowledge base before any fresh AI call. A suitable stored solution is retrieved and adapted; otherwise fresh AI debugging runs (see Figure 4).

The system must:

- Search knowledge before fresh AI calls.
- Reuse successful solutions.
- Send only relevant repository files.
- Cache reusable output where appropriate.
- Track token/request usage.
- Save successful adaptations.
- Use multiple providers with failover.

17. Article Evolution Rules

When multiple successful solutions exist:

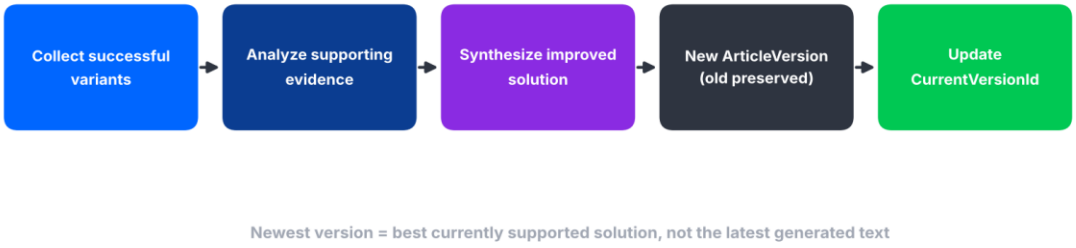


Figure 8 – Article evolution

1. Collect successful variants.
2. Analyze supporting evidence.
3. Synthesize improved solution.
4. Generate new ArticleVersion.
5. Preserve previous versions.
6. Update CurrentVersionId.

> **Principle**
> The newest version should represent the best currently supported solution, not simply the most recently generated text.

18. GitHub Challenge Flow

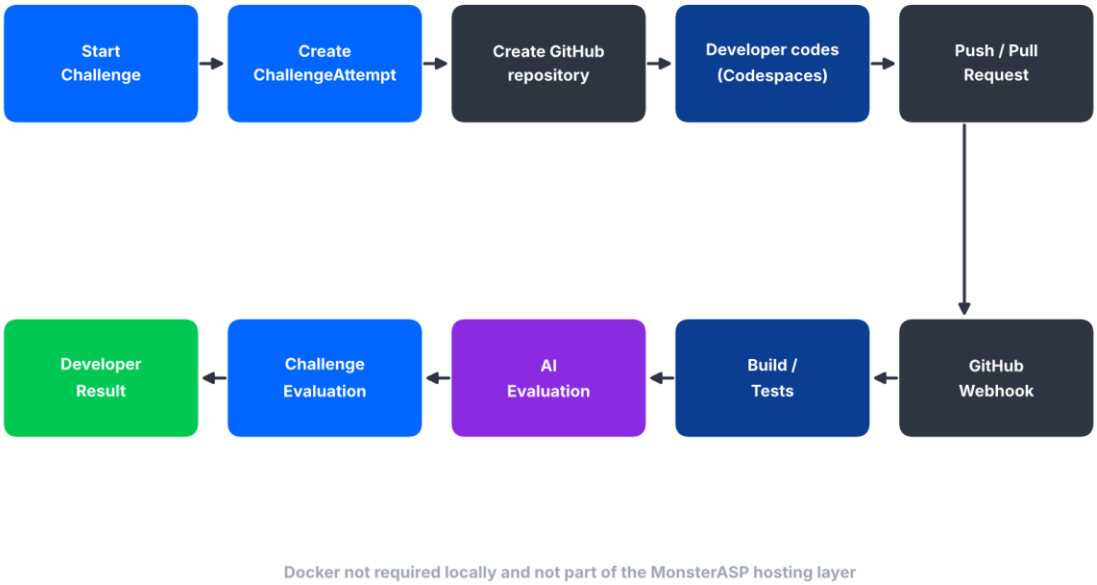


Figure 9 – Challenge lifecycle from start to developer result

Online development may use GitHub Codespaces / Dev Containers. Docker is not required on the developer's local machine and is not part of the MonsterASP hosting layer.

19. API Implementation Order



Figure 10 – Implementation roadmap

Phase	Scope
1	Authentication + Email Verification
2	Problems + Debugging + Feedback + History
3	Knowledge Base + Search + Similarity
4	AI Gateway + Provider Management + Usage
5	GitHub OAuth + Repository Access + Webhooks
6	Challenges + GitHub Challenge Repository + Evaluation
7	AI Code Review
8	Telegram
9	Admin

20. Final API Flow

The frontend calls the ASP.NET Core Web API, which exposes the Authentication, Problems, Debugging, Search / Similarity, Knowledge, Challenges, GitHub, Code Review, Telegram and Admin modules. These rely on the AI Gateway and the Email Service, which in turn connect to SQL Server, Brevo, GitHub, Telegram and the LLM providers.

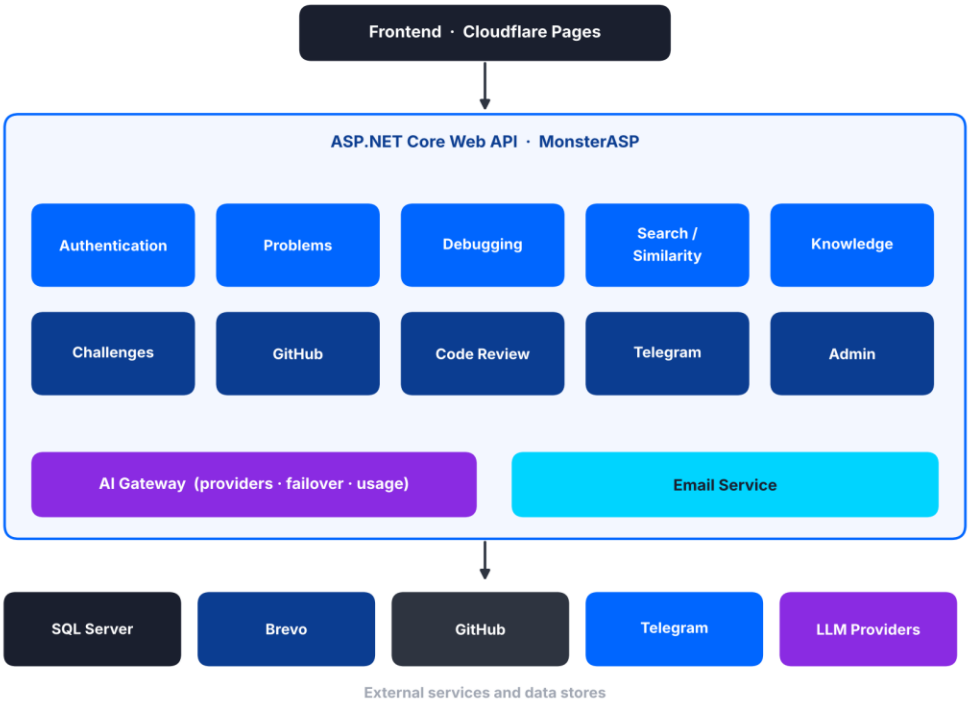


Figure 11 – Final API flow